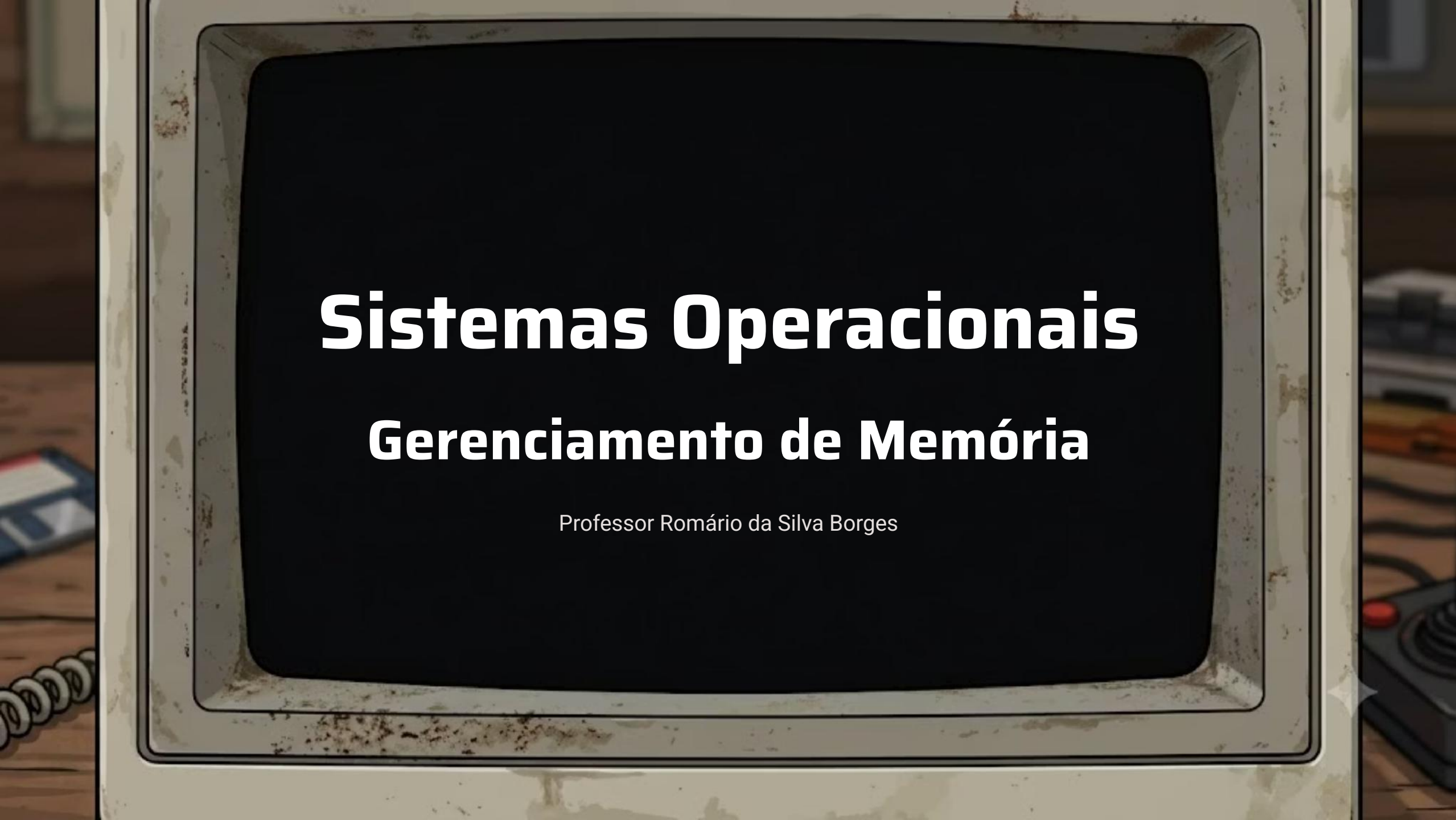




Sistemas Operacionais

Gerenciamento de Memória

Professor Romário da Silva Borges

**Espaço de endereçamento virtual
de um processo**

Espaço de endereçamento virtual de um processo

Na maioria dos sistemas atuais, o espaço de endereços virtuais de cada processo é organizado da seguinte forma: a parte inicial dos endereços virtuais é reservada para uso do processo, enquanto a parte final é reservada para o núcleo do sistema operacional.

Observa-se que o núcleo faz parte do espaço de endereçamento de cada processo, mas não é acessível ao mesmo, por conta dos flags de controle da tabela de páginas.

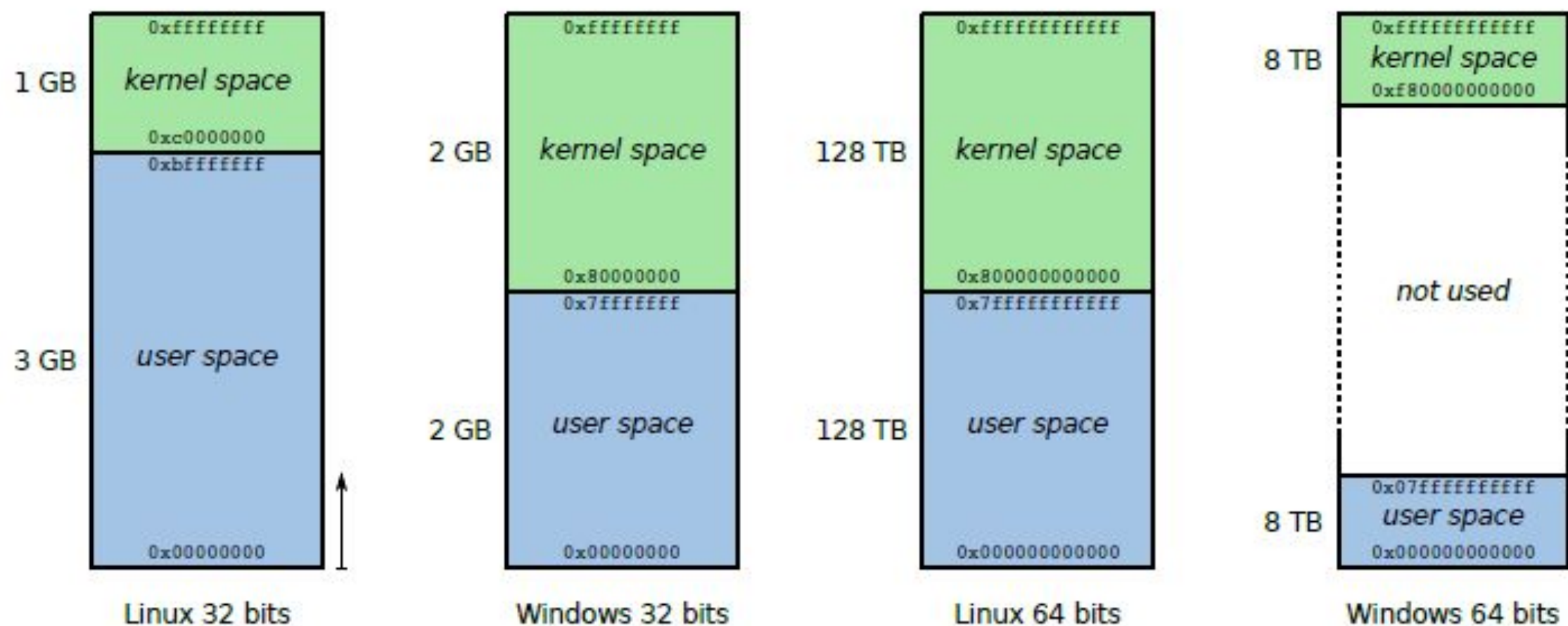


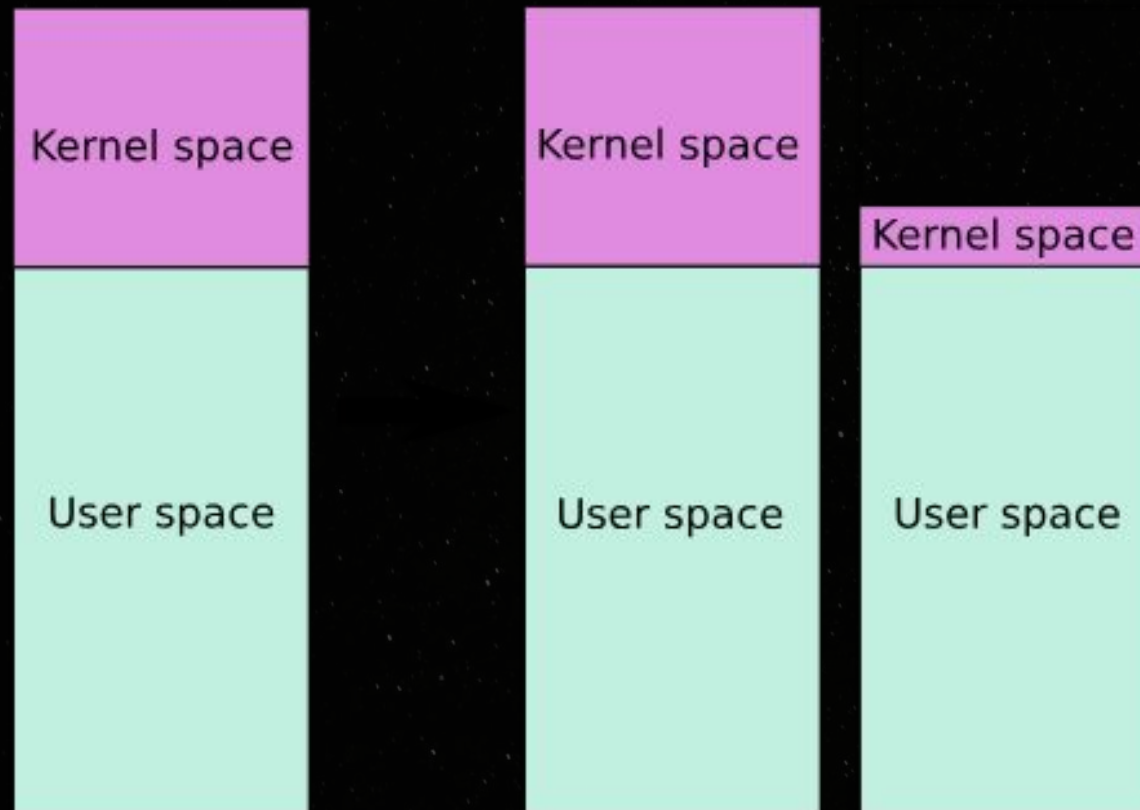
Figura 15.1: Organização do espaço de endereçamento em SOs atuais.

Bug de segurança *Meltdown*



<https://meltdownattack.com/>

Solução encontrada pelo Linux foi a **Kernel Page-Table Isolation (KPTI)** mantém apenas um pequeno conjunto de páginas do núcleo no espaço de endereços do processo, para receber syscalls e interrupções.



A memória de um processo

Cada processo é implementado pelo sistema operacional como uma “cápsula” de memória isolada dos demais processos, ou seja, uma área de memória exclusiva que só o próprio processo e o núcleo do sistema podem acessar. A área de memória de um processo contém todas as informações necessárias à sua execução, como:

- código binário do programa;
- variáveis;
- bibliotecas utilizadas;
- buffers;
- pilha (stack);
- dados alocados dinamicamente (heap);
- entre outras informações.

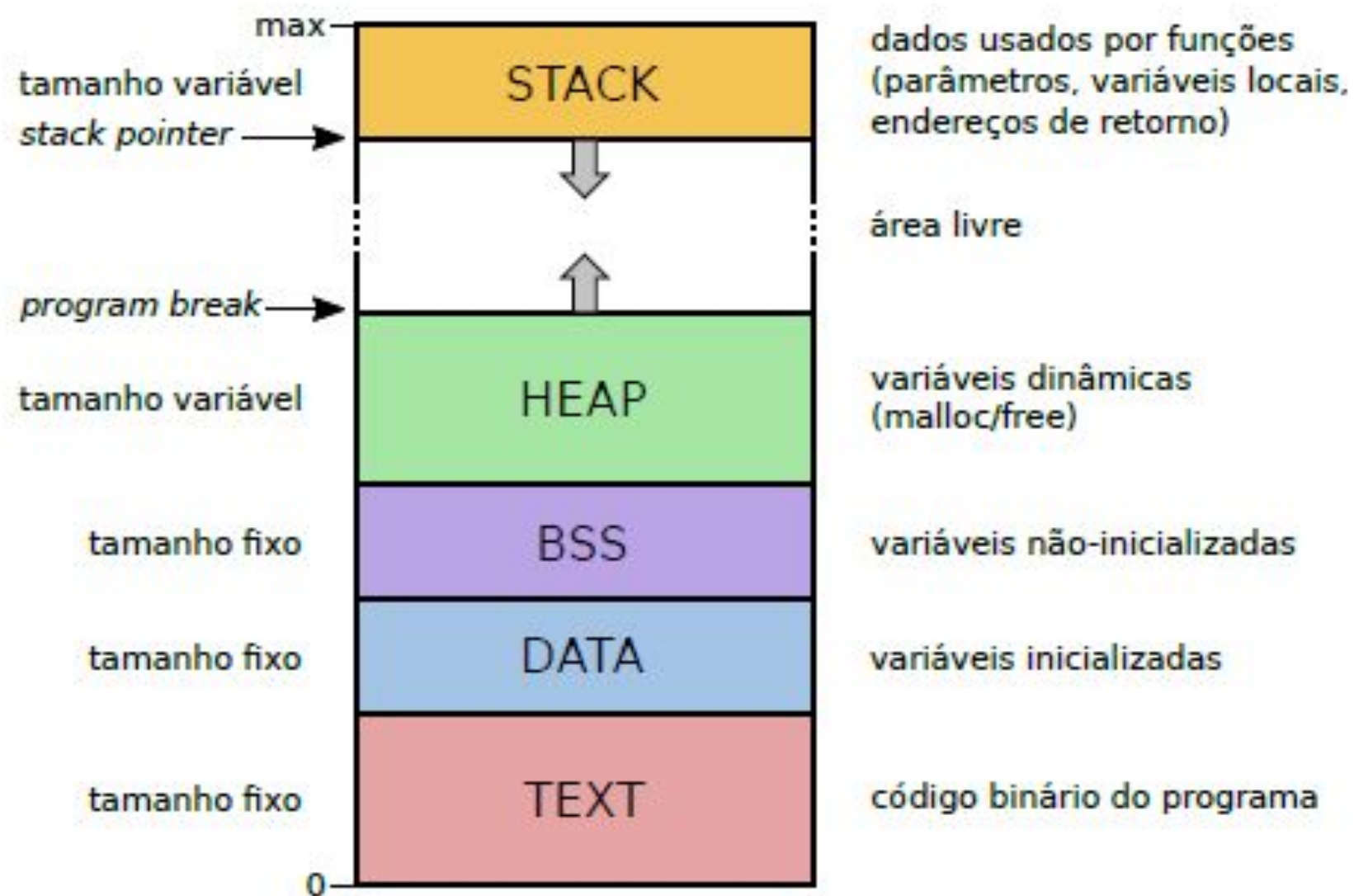


Figura 15.2: Organização da memória de um processo.

No sistema Linux, o comando `pmap` (*process map*) permite observar o mapa de memória de um processo, ou seja, a distribuição das seções de memória que o compõem. O exemplo a seguir ilustra o resultado (simplificado) desse comando aplicado a um processo que executa um simples `printf` ("Hello World\n") em linguagem C (27505 é o PID, identificador numérico do processo):

```
1 # pmap 27505
2
3 27505:  /usr/bin/hello
4 Address          Kbytes    Mode    Mapping
5 000000000000400000    808    r-x--  /usr/bin/hello    (TEXT)
6 0000000000006c9000     12    rw---  /usr/bin/hello    (DATA)
7 0000000000006cc000      8    rw---  [ anon ]          (BSS)
8 00000000000092e000    140    rw---  [ anon ]          (HEAP)
9 00007ffe6a5df000    132    rw---  [ stack ]         (STACK)
```

```
int soma = 0;  
int temp;  
  
int main() {  
    int i;  
    int* ptr = malloc(sizeof(int));  
}
```



Alocação de variáveis

Um programa em execução armazena suas informações em variáveis, que são basicamente espaços de memória reservados e nomeados. Por exemplo, uma declaração `int soma` em linguagem C indica uma área de memória de 4 bytes que pode armazenar um número inteiro e cujo nome é soma. Em linguagens como C e C++, variáveis podem ser alocadas na memória usando três abordagens usuais: a alocação estática, a alocação automática e a alocação dinâmica.

Alocação estática

```
1 #include <stdio.h>
2
3 int soma = 0 ;
4
5 int main ()
6 {
7     int i ;
8
9     for (i=0; i<1000; i++)
10         soma += i ;
11     printf ("Soma de inteiros até 1000: %d\n", soma) ;
12
13     return (0) ;
14 }
```

Alocação automática

```
1 #include <stdio.h>
2
3 long int fatorial (int n)
4 {
5     long int parcial ;
6
7     printf ("inicio: n = %d\n", n) ;
8     if (n < 2)
9         parcial = 1 ;
10    else
11        parcial = n * fatorial (n - 1) ;
12    printf ("final : n = %d, parcial = %ld\n", n, parcial) ;
13    return (parcial) ;
14 }
15
16 int main ()
17 {
18     printf ("Fatorial (4) = %ld\n", fatorial (4)) ;
19     return 0 ;
20 }
```

Alocação dinâmica

```
1 char * prt ;           // ponteiro para caracteres
2
3 ptr = malloc (4096) ;   // solicita um bloco de 4.096 bytes;
4                          // ptr aponta para o início do bloco
5
6 if (!ptr)               // se ptr for nulo, ocorreu um erro
7     abort () ;          // e a área não foi alocada
8
9 ...                     // usa ptr para acessar o bloco alocado
10
11 free (ptr) ;           // libera o bloco alocado na linha 3
```

```
1 Rectangle rect1 = new Rectangle (10, 30) ;
2 Rectangle rect2 = new Rectangle (3, 2) ;
3 Triangle tr1 = new Triangle (3, 4, 5) ;
```

	Estática	Automática	Dinâmica
Onde Fica?	BSS / Data	Stack	Heap
Ciclo de Vida	Toda a execução da tarefa 	Apenas durante o escopo { } 	Controle explícito (malloc/free) 
Gerenciamento	Automático (Compilador)	Automático (Compilador)	Manual (Programador)
Tamanho	Fixo na compilação	Fixo na chamada	Totalmente redimensionável

Atribuição de endereços

Ao escrever um programa usando uma linguagem de programação, como C,C++ ou Java, o programador usa nomes para referenciar entidades abstratas como variáveis, funções, parâmetros e valores de retorno. Com esses nomes, não há necessidade do programador definir ou manipular endereços de memória explicitamente.

```
1 #include <stdlib.h>
2 #include <stdio.h>
3
4 int soma = 0 ;
5
6 int main ()
7 {
8     int i ;
9
10    for (i=0; i< 10; i++)
11    {
12        soma += i ;
13        printf ("i vale %d e soma vale %d\n", i, soma) ;
14    }
15    exit(0) ;
16 }
```

O código Assembly correspondente à compilação do programa em linguagem C soma.c, têm-se que os nomes simbólicos das variáveis e blocos de código usados por um programa devem ser traduzidos em endereços de memória em algum momento entre a escrita do código pelo programador e sua execução pelo processador.

```
1 0000000000000000 <main>:
2 0: 55          push  %rbp
3 1: 48 89 e5     mov   %rsp,%rbp
4 4: 48 83 ec 10   sub   $0x10,%rsp
5 8: c7 45 fc 00 00 00 00 movl  $0x0,-0x4(%rbp)
6 f: eb 2f        jmp   40 <main+0x40>
7 11: 8b 15 00 00 00 00 mov   0x0(%rip),%edx
8 17: 8b 45 fc      mov   -0x4(%rbp),%eax
9 1a: 01 d0        add   %edx,%eax
10 1c: 89 05 00 00 00 00 mov   %eax,0x0(%rip)
11 22: 8b 15 00 00 00 00 mov   0x0(%rip),%edx
12 28: 8b 45 fc      mov   -0x4(%rbp),%eax
13 2b: 89 c6        mov   %eax,%esi
14 2d: bf 00 00 00 00 mov   $0x0,%edi
15 32: b8 00 00 00 00 mov   $0x0,%eax
16 37: e8 00 00 00 00 callq 3c <main+0x3c>
17 3c: 83 45 fc 01   addl  $0x1,-0x4(%rbp)
18 40: 83 7d fc 04   cmpl  $0x4,-0x4(%rbp)
19 44: 7e cb        jle   11 <main+0x11>
20 46: bf 00 00 00 00 mov   $0x0,%edi
21 4b: e8 00 00 00 00 callq 50 <main+0x50>
```

Tradução de endereços

A atribuição de endereços aos nomes simbólicos pode ocorrer em diversos momentos da vida do programa:

- **Na edição (Codificação):** O próprio programador define manualmente os endereços de memória das variáveis e do código. É comum apenas em sistemas embarcados simples, geralmente em Assembly.
- **Na compilação:** O compilador já define os endereços durante a tradução do código. Por isso, precisa conhecer todo o código do programa para evitar conflitos de endereços.

- **Na ligação (linking):** O compilador gera arquivos-objeto sem definir todos os endereços finais. O linker combina esses arquivos, resolve os símbolos e define seus endereços, gerando o executável.
- **Na carga (loading):** Os endereços são definidos quando o programa é carregado na memória para ser executado. O loader realiza essa tarefa, sendo também importante no carregamento de bibliotecas dinâmicas (DLLs).
- **Na execução:** Os endereços gerados pelo processador são convertidos durante a própria execução em endereços da memória física. Essa tradução ocorre continuamente com auxílio do hardware, como a MMU.

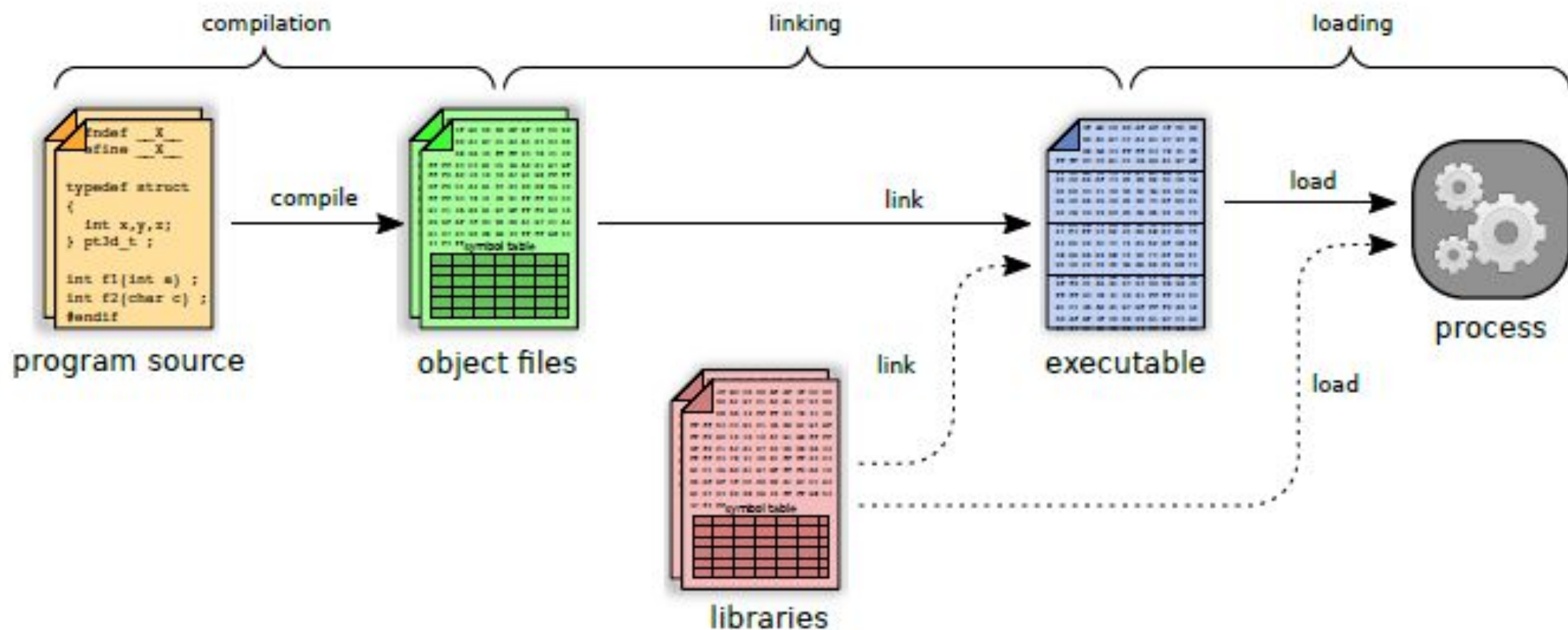


Figura 15.3: Momentos da tradução de endereços.

1. Código-Fonte em C

```
</> _____  
_____  
  
int x = 10;  
char *p = "hello";  
  
void foo() {  
    ...  
    _____  
    _____  
}  
}
```

Nomes simbólicos

2. Compilação e Ligação



Compilador



Linker

Tabela de Símbolos

Offset	
x:	Offset 0x1000
p:	Offset 0x1010
foo:	Offset 0x2000

Endereços Relativos Fixos

3. Carregamento na Memória RAM



0x1000: [0A 00 00 00] (Valor 10)

0x1010: [68 65 6C 6C 6F] ("hello")

0x2000: [55 48 89 E5 ...] (foo())

...

Memória RAM
(Endereços Reais)

Execução
Direta



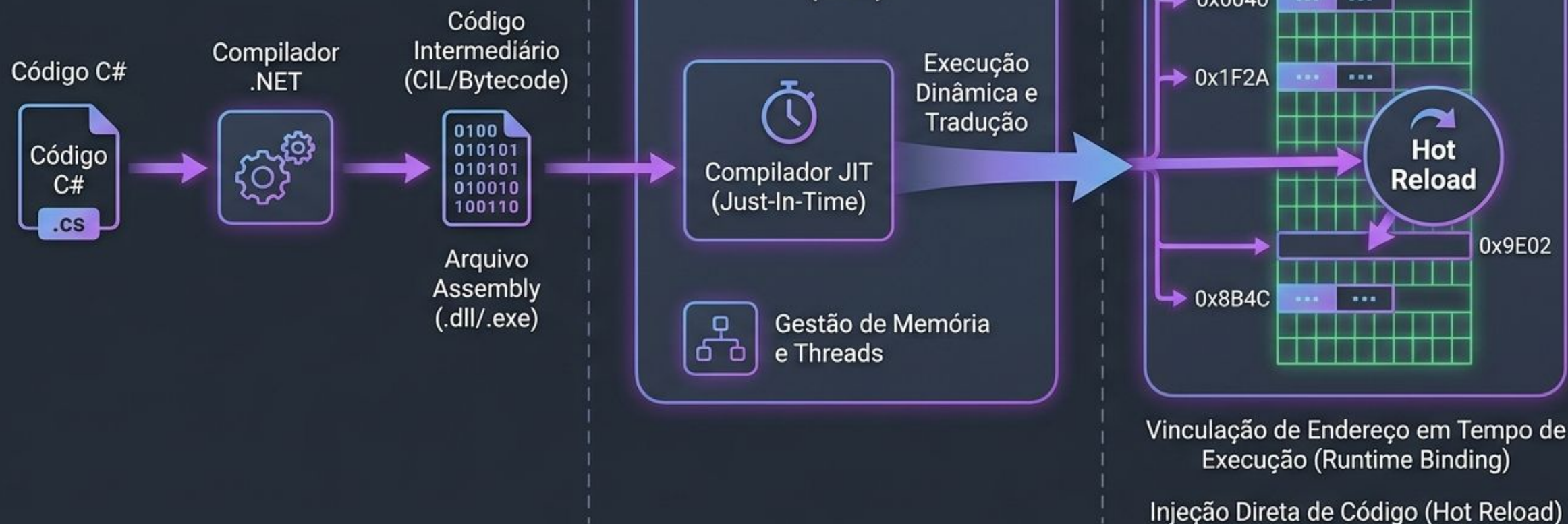
Atribuição
Estática
de
Endereço

Arquitetura de Vinculação Dinâmica de Endereço (JIT) no .NET

1. Código Fonte e Compilação Estática

2. Ambiente de Execução e Compilação Dinâmica

3. Alocação Dinâmica e Vinculação de Endereço na RAM



- Código-Fonte C#: É traduzido primeiro para uma linguagem intermediária (CIL / Bytecode).
- Ambiente de Execução (CLR & JIT): O compilador JIT (Just-In-Time) converte as instruções em código de máquina nativo e atribui os endereços de memória na RAM somente no instante em que o código é executado.
- Alocação Dinâmica: Esse adiamento da atribuição para o tempo de execução permite recursos como o Hot Reload, atualizando o código em memória sem parar o processo.

Referências bibliográficas

TANENBAUM, Andrew S.; BOS, Herbert.

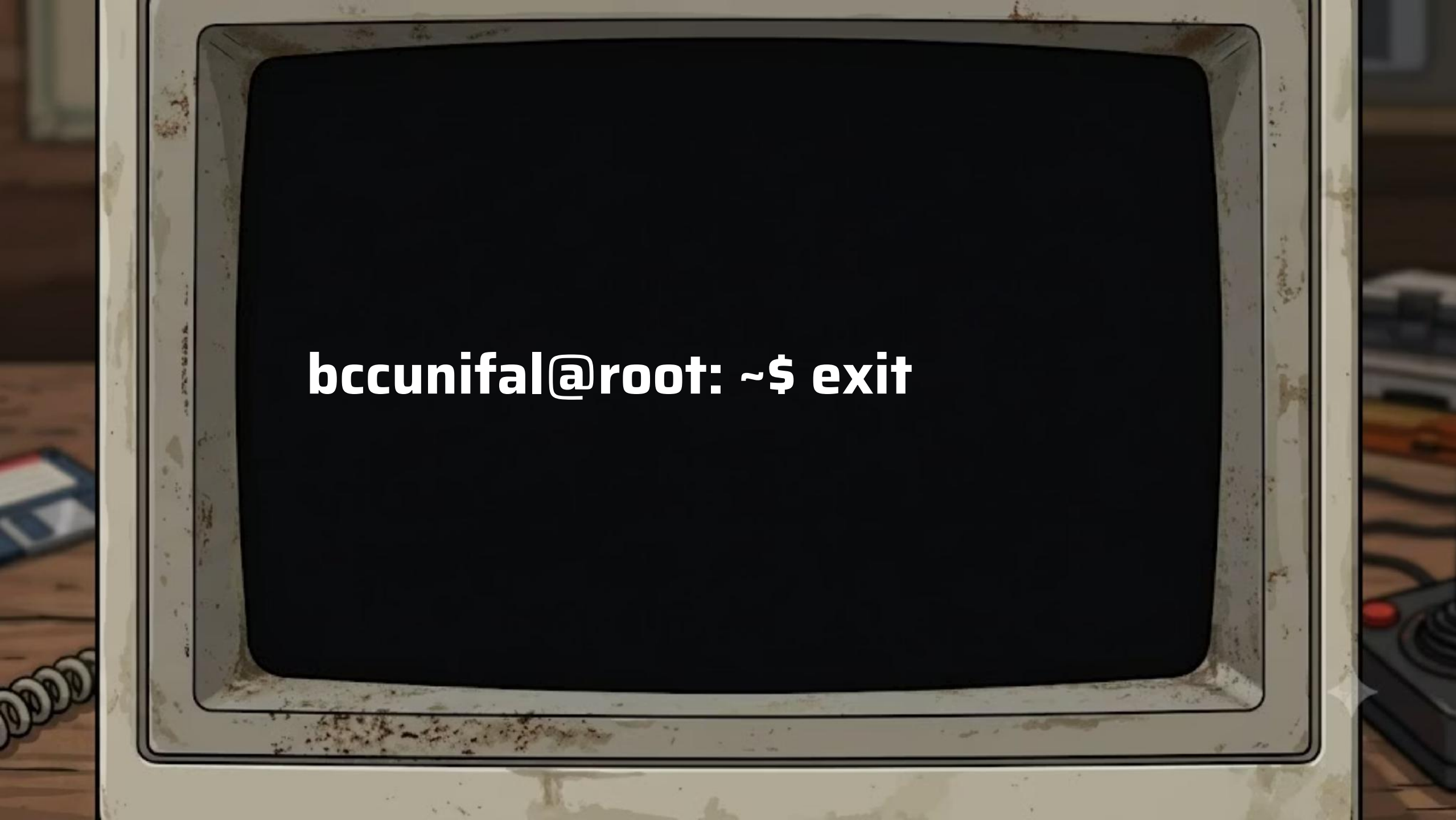
Sistemas Operacionais Modernos. 4. ed. São Paulo: Pearson, 2016.

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg.

Fundamentos de Sistemas Operacionais. 9. ed. Rio de Janeiro: LTC, 2015.

MAZIERO, Carlos A.

Sistemas Operacionais: Conceitos e Mecanismos. Curitiba: UFPR, 2019.

A terminal window is displayed on a computer monitor. The monitor has a light-colored, slightly textured bezel. The terminal window itself is a dark rectangle with a thin white border. Inside the terminal, the text 'bccunifal@root: ~\$ exit' is written in a white, monospaced font. The background of the entire image shows a desk with a keyboard and some papers, though they are out of focus.

bccunifal@root: ~\$ exit